RAPPORT DE STAGE

Méthode de Monte Carlo pour l'étude des transferts thermiques

Du 4 mai au 12 juin

Stagiaire : David Cabanillas

Maître de stage : Stéphane Blanco, enseignant-chercheur

Laboratoire LAPLACE

Bâtiment 3R1, 325 rue Sébastienne Guyot, Université Paul Sabatier
31400 Toulouse

Sommaire

I Abstract 4

II Résumé 4

III Introduction 5

III.1 Le problème de départ 5

III.2 La solution étudiée : Monte Carlo 5

III.3 Les deux objectifs de mon stage 5

III.4 Plan du rapport 6

IV Le Laplace 6

IV.1 Présentation générale 6

IV.2 Mon équipe 7

V Méthode de Monte Carlo pour l'étude des transferts thermiques 8

V.1 Comprendre la méthode de Monte Carlo 8

V.1.a L'idée en une image 8

V.1.b L'équation de départ : un fluide chauffé par ses parois 8

V.1.c Faire apparaître une moyenne 9

V.1.d L'algorithme : un chemin thermique qui remonte le temps 9

V.1.e Une astuce de calcul : la méthode de rejet 10

V.1.f Conduction, rayonnement et passages entre milieux 10

V.2 Prise en main des outils 11

V.2.a L'environnement Linux 11

V.2.b Réalisation et modification des Travaux Pratiques 12

V.3 Comment le code de Stardis est organisé 12

V.3.a Deux programmes : Stardis et le solveur 12

V.3.b Des « fiches » qui décrivent le chemin : les structures 12

V.3.c Une seule écriture pour la 2D et la 3D : la macro XD 13

V.3.d Le chef d'orchestre : la boucle de compute_temperature 13

V.4 Hack de Stardis : la conduction dans un fluide 14

V.4.a Comment indiquer à Stardis qu'un fluide conduit la chaleur ? 14

V.4.b Vue d'ensemble 14

V.4.c Étape A : détecter le code dans la fonction de convection 14

V.4.d Étape B : une fonction de conduction adaptée aux fluides 16

V.4.e Étape C : la réinjection, la partie la plus longue 17

V.4.f Le dernier bug : un « point de contact » oublié 17

V.4.g Bilan de l'étape 1 et limites 18

V.5 Conseils pour les futurs arrivants sur Stardis 18

VI Enjeux sociétaux et environnementaux 19

VII Analyse réflexive 20

VII.1 Au cours de mon stage 20

VII.2 Au sein de mon projet professionnel 21

VIII Conclusion 21

IX Remerciements 21

X Références Bibliographiques 22

XI Annexes 23

XI.1 Liste des tâches effectuées 23

XI.2 Glossaire et nomenclature 23

XI.3 Fiche de synthèse 25

XI.4 Notions du langage C 25

XI.5 Les macros dans Stardis 26

XI.6 Le code ajouté 27

I Abstract

This internship focused on simulating heat transfer with Monte Carlo methods. Instead of solving the heat equations on a grid, these methods follow many random “thermal paths” and average their results. Combined with ray-tracing techniques from computer graphics, they can handle very large and very detailed geometries, such as a whole city for urban heat island studies.

The GREPHE research group at the LAPLACE laboratory uses a Monte Carlo heat transfer software called Stardis. Some physical situations of interest cannot yet be simulated with it. My main task was to modify Stardis so that thermal paths can travel by conduction inside a fluid, a first step towards modelling snow and water vapour. I also documented the difficulties a newcomer faces when learning the Stardis code.

II Résumé

Ce stage porte sur la simulation des transferts de chaleur par la méthode de Monte Carlo. Au lieu de résoudre les équations de la chaleur sur un maillage, cette méthode suit un grand nombre de « chemins thermiques » tirés au hasard et fait la moyenne de leurs résultats. Associée aux techniques de lancer de rayons de l’informatique graphique, elle peut traiter des géométries à la fois très grandes et très détaillées, comme une ville entière pour étudier les îlots de chaleur urbains.

Le groupe GREPHE du laboratoire LAPLACE utilise un logiciel de ce type, appelé Stardis. Certaines situations physiques ne peuvent pas encore y être simulées. Ma mission principale a été de modifier Stardis pour que les chemins thermiques puissent se propager par conduction dans un fluide, première étape vers la modélisation de la neige et de la vapeur d’eau. J’ai aussi noté les difficultés rencontrées par un nouvel arrivant qui découvre le code de Stardis.

III Introduction

En une phrase : j'ai modifié Stardis, un logiciel qui calcule des températures par tirages au hasard, pour qu'il sache simuler la conduction de la chaleur dans un fluide, ce qu'il ne savait pas faire.

III.1 Le problème de départ

Prévoir la température d'un objet ou d'un lieu est utile partout : isoler un bâtiment, refroidir une batterie, comprendre pourquoi une ville est plus chaude que la campagne. La chaleur s'y déplace de trois façons :

- **la conduction** : la chaleur traverse la matière, comme dans une cuillère posée dans un café chaud ;
- **la convection** : un fluide (air, eau) en mouvement emporte la chaleur, comme l'air d'un radiateur ;
- **le rayonnement** : la chaleur voyage sous forme de lumière, même invisible, comme celle du soleil.

Les méthodes de calcul classiques découpent l'espace en petites cases et calculent la température case par case. Quand la géométrie est à la fois immense et très détaillée (une ville avec ses fenêtres et ses murs), le nombre de cases explose et le calcul devient trop long.

III.2 La solution étudiée : Monte Carlo

La méthode de Monte Carlo procède autrement. Pour connaître la température en un point, elle lance un grand nombre de **chemins thermiques** aléatoires depuis ce point. Chaque chemin « remonte le temps » en passant d'un mode de transfert à l'autre (conduction, convection, rayonnement), jusqu'à tomber sur un endroit où la température est déjà connue. La moyenne des températures trouvées au bout de tous les chemins donne la température cherchée. La partie V.1 explique pourquoi cela fonctionne.

Le laboratoire utilise pour cela le logiciel **Stardis**, développé par l'entreprise Méso-Star.

III.3 Les deux objectifs de mon stage

1. **Faire évoluer Stardis.** Deux collègues stagiaires étudient un milieu composé de neige et de vapeur d'eau. Les poches de vapeur y sont trop petites pour que l'air circule : la chaleur n'y passe pas par convection mais par conduction. Or Stardis ne sait faire que de la convection dans un fluide. Le travail se découpe en deux étapes :
    a. permettre aux chemins thermiques de passer en conduction dans un fluide (étape réalisée, c'est le cœur de ce rapport) ;

   b. ajouter un nouveau mode de transfert dit « en humidité » dans la vapeur d'eau (commencé en fin de stage).

2. **Raconter la prise en main du code.** Seuls ses deux concepteurs, Vincent Forest et Christophe Coustet, connaissent vraiment le code de Stardis. Mon second objectif était de noter les difficultés d'un nouvel arrivant et d'en tirer des conseils pour les suivants (partie V.5).

Ces modifications sont un **prototype** : elles servent à tester si le modèle donne des résultats cohérents. Si c'est le cas, les développeurs de Méso-Star pourront l'intégrer proprement dans une future version.

III.4 Plan du rapport

Après une présentation du laboratoire (partie IV), j'explique simplement la méthode de Monte Carlo (partie V.1), puis les outils que j'ai appris (partie V.2) et l'organisation du code de Stardis (partie V.3). La partie V.4 décrit ma modification, la partie V.5 donne des conseils aux futurs arrivants. Je termine par les enjeux de ce travail et un bilan personnel. Un **glossaire** en annexe (XI.2) définit les mots techniques.

IV Le Laplace

IV.1 Présentation générale

Le LAPLACE (Laboratoire Plasma et Conversion d'Énergie) est l'un des plus grands centres français de recherche en génie électrique, plasmas et conversion d'énergie. Créé en 2007 par la fusion de plusieurs laboratoires, il dépend de trois tutelles : le CNRS, Toulouse INP et l'Université Toulouse III – Paul Sabatier. Il est installé sur deux sites toulousains : le campus de Paul Sabatier et l'ENSEEIHT.

Ses recherches vont de la physique fondamentale (plasmas, matériaux, électromagnétisme, thermique) jusqu'à la conception de systèmes complets de production et de conversion d'énergie. Il travaille avec de nombreux industriels dans l'aéronautique, le spatial, les transports, l'énergie, l'environnement et la santé.

Le laboratoire compte **12 groupes de recherche** (tableau 1).

Thème	Groupes
Plasmas et arcs électriques	AEPPT, PRHE, SCIPRA
Systèmes électriques et leur commande	CODIASE, GENESYS
Électromagnétisme et électronique de puissance	GRE, GREM3, CS
Matériaux diélectriques et fiabilité	DSF, MDCE
Phénomènes fondamentaux (lumière, matière, hors équilibre)	LM, GREPHE

Tableau 1 : Les groupes de recherche du LAPLACE

IV.2 Mon équipe

J'ai été accueilli au **GREPHE** (Groupe de Recherche Énergétique, Plasmas et Hors Équilibre), le plus grand groupe du LAPLACE. Il étudie des systèmes « hors équilibre », c'est-à-dire où l'énergie n'est pas répartie uniformément. Il réunit deux communautés :

- les spécialistes des **transferts thermiques** et des systèmes à deux phases (liquide et gaz) : conduction, convection, rayonnement, changements d'état ;
- les spécialistes des **plasmas froids** : physique des plasmas, propulsion spatiale, sources d'ions.

Ces deux communautés partagent les mêmes outils mathématiques, notamment la physique statistique et les méthodes de Monte Carlo. Le tableau 2 présente les personnes avec qui j'ai travaillé.

Personne	Rôle	Ce qu'elle m'a apporté
Stéphane Blanco	Enseignant-chercheur, maître de stage	Missions, suivi, notions de physique
Richard Fournier	Enseignant-chercheur	Suivi au quotidien, formation à Monte Carlo
Vincent Forest	Développeur chez Méso-Star	Langage C, fonctionnement de Stardis
Christophe Coustet	Développeur chez Méso-Star	Programmation, fonctionnement de Stardis
Mégane Bati	Chercheuse CNRS	Conception des TP Stardis, aide pour les réaliser

Tableau 2 : Mon équipe d'encadrement

V Méthode de Monte Carlo pour l'étude des transferts thermiques

V.1 Comprendre la méthode de Monte Carlo

Pour modifier Stardis, il faut d'abord comprendre ce qu'il calcule. Cette partie montre, sur l'exemple de la convection, comment une équation de physique se transforme en une suite de tirages au hasard.

V.1.a L'idée en une image

Imaginons qu'on veuille connaître la moyenne d'un dé. On peut la calculer (3,5), mais on peut aussi lancer le dé 10 000 fois et faire la moyenne des résultats : on trouvera presque exactement 3,5. C'est le principe de Monte Carlo : **une moyenne peut être estimée en répétant une expérience aléatoire un grand nombre de fois**.

L'astuce de la thermique par Monte Carlo est de réécrire les équations de la chaleur pour que la température apparaisse comme une moyenne. Il suffit alors d'inventer l'expérience aléatoire correspondante, et de la répéter : chaque répétition est un **chemin thermique**.

V.1.b L'équation de départ : un fluide chauffé par ses parois

On considère un volume de fluide Ω (par exemple l'air d'une pièce), bien mélangé, donc à une température θ_F uniforme. Il est entouré de parois solides de température θ_S. Le fluide échange de la chaleur avec les parois par convection, avec un coefficient d'échange h qui peut varier d'un endroit à l'autre de la paroi. Les notations sont rappelées en annexe XI.2.

La variation d'énergie du fluide est égale à la chaleur reçue à travers ses parois (loi de convection) :

$$\rho c V \frac{d\theta_F}{dt} = \int_{\partial\Omega} h(\vec{x}) \left(\theta_S(\vec{x},t) - \theta_F(t)\right) d\vec{x}$$

On définit deux grandeurs pour simplifier l'écriture :

$$\nu = \frac{\int_{\partial\Omega} h\, d\vec{x}}{\rho c V} \qquad\qquad \theta_S^*(t) = \int_{\partial\Omega} \frac{h(\vec{x})}{\int_{\partial\Omega} h\, d\vec{x}}\, \theta_S(\vec{x},t)\, d\vec{x}$$

- ν (en s^{-1}) mesure la vitesse à laquelle le fluide s'adapte à ses parois.
- θ_S^* est la **température moyenne des parois, pondérée par** h : les zones où l'échange est fort comptent davantage, ce qui est intuitif.

L'équation devient :

$$\frac{d\theta_F}{dt} = -\nu\,\theta_F + \nu\,\theta_S^*$$

V.1.c Faire apparaître une moyenne

La solution de cette équation différentielle s'écrit :

$$\theta_F(t) = \theta_F(0)\, e^{-\nu t} + \int_0^t \nu e^{-\nu t'}\, \theta_S^*(t-t')\, dt'$$

Avec la fonction de Heaviside H (qui vaut 1 si son argument est positif, 0 sinon), on peut regrouper les deux termes dans une seule intégrale :

$$\theta_F(t) = \int_0^{+\infty} \nu e^{-\nu t'} \Big[H(t'-t)\, \theta_F(0) + H(t-t')\, \theta_S^*(t-t') \Big]\, dt'$$

Or la fonction $\nu e^{-\nu t'}$ a une intégrale égale à 1 entre 0 et l'infini : c'est la densité de probabilité d'une **loi exponentielle**. L'égalité ci-dessus se lit donc ainsi :

> *La température du fluide à l'instant t est la **moyenne** de la quantité entre crochets, quand on tire une durée t' au hasard selon une loi exponentielle.*

De même, θ_S^* est une moyenne de θ_S quand on tire un point de la paroi au hasard, avec une probabilité proportionnelle à h.

V.1.d L'algorithme : un chemin thermique qui remonte le temps

On en déduit l'expérience aléatoire à répéter. Pour estimer la température du fluide à l'instant t :

1. Tirer une durée t' selon une loi exponentielle de paramètre ν.
2. **Si** $t' > t$: on est remonté avant le début de la simulation. Le chemin a atteint la **condition initiale** : le résultat est $\theta_F(0)$, qui est connu. Le chemin s'arrête.
3. **Si** $t' < t$: le chemin se déplace à l'instant $t - t'$ sur la paroi. On tire un point de la paroi au hasard, en favorisant les zones où h est grand. La température cherchée devient celle du solide en ce point, à l'instant $t - t'$.
4. Cette température du solide est inconnue : on **continue le chemin** dans le solide (par conduction), puis éventuellement dans d'autres milieux, jusqu'à atteindre une température connue (condition initiale ou condition aux limites).
5. Recommencer des milliers de fois, puis faire la moyenne des températures trouvées au bout des chemins.

Deux remarques rendent cet algorithme puissant :

- **Une moyenne de moyennes reste une moyenne.** On peut donc enchaîner les étapes aléatoires d'un milieu à l'autre sans jamais calculer les températures intermédiaires.
- **Les chemins remontent le temps.** On peut calculer une température à un instant

très lointain sans calculer toutes les températures intermédiaires, contrairement aux méthodes classiques qui avancent pas à pas.

V.1.e Une astuce de calcul : la méthode de rejet

Tirer directement un point de la paroi avec une probabilité proportionnelle à h est coûteux en calcul : il faudrait découper la paroi en morceaux et en tenir compte à chaque tirage. Stardis contourne le problème avec une **méthode de rejet**.

On choisit un taux $\hat{\nu}$ plus grand que ν, et on réécrit l'équation sans la changer :

$$\frac{d\theta_F}{dt} = -\hat{\nu}\,\theta_F + \hat{\nu}\left[\left(1-\frac{\nu}{\hat{\nu}}\right)\theta_F + \frac{\nu}{\hat{\nu}}\,\theta_S^*\right]$$

Dans l'algorithme, cela revient à :

1. tirer t' avec le taux $\hat{\nu}$ (plus grand, donc des durées plus courtes) ;
2. si $t' < t$, faire un « pile ou face » truqué, qui réussit avec la probabilité $\nu/\hat{\nu}$;
3. en cas de **succès**, aller sur la paroi comme avant ;
4. en cas d'**échec**, rester dans le fluide, à l'instant $t - t'$, et recommencer au point 1.

Le résultat moyen est exactement le même, mais chaque tirage est beaucoup plus simple à réaliser.

V.1.f Conduction, rayonnement et passages entre milieux

Les chemins de conduction et de rayonnement reposent sur le même principe, mais se déplacent aussi dans l'espace : en conduction, le chemin avance par petits sauts aléatoires dans la matière, un peu comme une particule qui diffuse. Quand un chemin arrive à une **interface** entre deux milieux, il tire au hasard le milieu dans lequel il repart et son mode de transfert (conduction, convection ou rayonnement), avec des probabilités fixées par la physique de l'interface. La figure 1 montre un tel chemin.

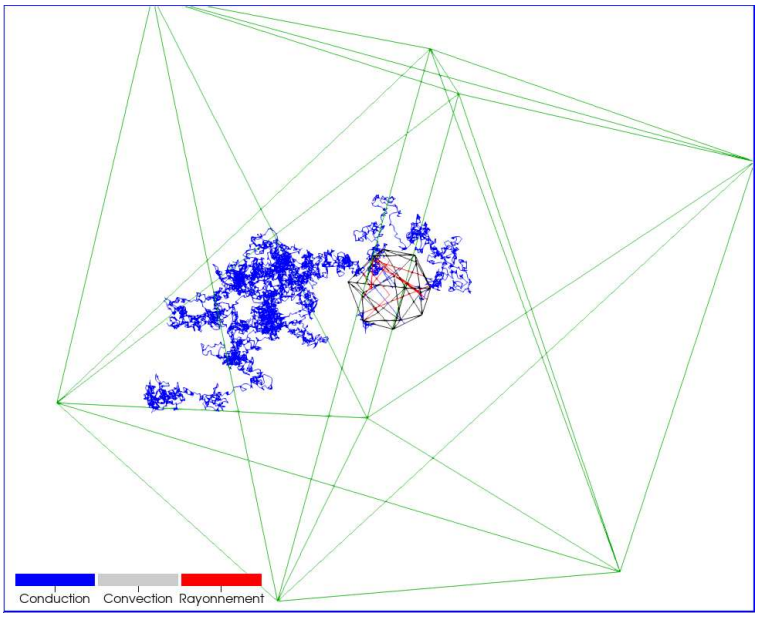


Figure 1 : Un chemin thermique visualisé dans Paraview : en bleu la conduction, en gris la convection, en rouge le rayonnement

V.2 Prise en main des outils

Avant de toucher au code, j'ai dû apprendre l'environnement de travail du laboratoire, puis utiliser Stardis à travers des travaux pratiques (TP).

V.2.a L'environnement Linux

Tous les chercheurs travaillent sous **Linux**, avec une version très dépouillée appelée **edix** : un bureau à 9 espaces de travail, un terminal et le navigateur Firefox. Presque tout se fait en tapant des commandes dans le terminal. J'ai appris les outils du tableau 3 (la commande `man edix` donne une aide).

Outil	**À quoi il sert**
`ls`, `cd`	Lister les fichiers, changer de dossier
`cp`, `mv`, `rm`	Copier, renommer ou déplacer, supprimer un fichier
`mount /dev/sdy1 <dossier>`	Rendre le contenu de ma clé USB accessible dans un dossier, pour retrouver mon travail sur n'importe quel poste
`vim`	Éditeur de texte dans le terminal, indispensable pour les TP et le code
`git`	Garder l'historique de toutes les versions d'un projet, avec un commentaire pour chaque modification

Tableau 3 : Commandes et outils appris

V.2.b Réalisation et modification des Travaux Pratiques

TP 1 : lancer Stardis. J'ai appris à lancer un calcul en ligne de commande (figure 2).

```
stardis -V 3 -M model.txt -p 0.5,0.5,0.5 -n 10000 -e
```

Figure 2 : Exemple de commande Stardis

Ici, `-M model.txt` donne le fichier qui décrit les matériaux et la géométrie, `-p 0.5,0.5,0.5` le point où l'on veut la température, et `-n 10000` le nombre de chemins thermiques. J'ai aussi appris à visualiser les chemins avec le logiciel **Paraview** (figure 1).

Amélioration du TP 1. Les TP, écrits par une seule personne et peu relus, contenaient des consignes peu claires et des erreurs. J'ai réécrit le TP 1 pour qu'un débutant puisse le suivre. J'ai travaillé sur une copie du dépôt obtenue avec `git clone`, pour que chaque modification soit tracée.

TP 6 : une première modification guidée du code. Ce TP fait ajouter une variable dans le code de Stardis pour changer le calcul de la puissance volumique (la chaleur produite par unité de volume). J'y ai compris qu'ajouter une variable oblige à modifier plusieurs fonctions, car chaque variable a ses propres fonctions d'accès.

V.3 Comment le code de Stardis est organisé

Stardis est écrit en **langage C**, que j'ai dû apprendre pendant le stage (structures, pointeurs, macros, organisation d'un projet en plusieurs fichiers ; voir l'annexe XI.4). Cette partie donne les quatre idées à connaître pour s'y retrouver.

V.3.a Deux programmes : Stardis et le solveur

- **Stardis** lit la commande tapée dans le terminal et le fichier de description du modèle (point de calcul, nombre de chemins, matériaux…).
- **Stardis-solveur** fait le calcul : il trace les chemins thermiques et estime la température.

Toutes mes modifications ont été faites dans **Stardis-solveur**.

V.3.b Des « fiches » qui décrivent le chemin : les structures

Une **structure** en C regroupe plusieurs variables sous un même nom, comme une fiche qui rassemble des informations liées. La plus importante pour moi est `rwalk` (*random walk*, marche aléatoire), qui décrit l'état courant d'un chemin thermique (figure 3).

```c
struct rwalk {
  struct sdis_rwalk_vertex vtx; /* Position et instant du chemin */
  unsigned enc_id;              /* Enceinte où se trouve le chemin */
  struct s2d_hit hit_2d;        /* Dernier point de paroi touché (2D) */
  struct s3d_hit hit_3d;        /* Dernier point de paroi touché (3D) */
  double dir[3];                /* Direction vers le rayonnement */
  double elapsed_time;
  enum sdis_side hit_side;      /* Côté de la paroi touché */
};
```

Figure 3 : Définition de la structure rwalk

Toutes les fonctions qui construisent un chemin reçoivent les mêmes cinq structures : la scène (`sdis_scene`, la géométrie), le contexte (`rwalk_context`), l'état du chemin (`rwalk`), le générateur de nombres aléatoires (`ssp_rng`) et la température en cours de calcul (`temperature`).

Pour lire une propriété, le code passe toujours par des fonctions « get » (par exemple `fluid_get_temperature`). Elles gèrent le cas où la valeur n'est pas définie (valeur `NULL`), ce qui rend le code plus robuste.

V.3.c Une seule écriture pour la 2D et la 3D : la macro XD

Stardis fonctionne en 2D et en 3D. Les deux versions d'une fonction sont presque identiques. Plutôt que de tout écrire deux fois, les développeurs écrivent une seule fonction `XD(nom)`. Avant la compilation, une **macro** (une règle de remplacement de texte) en fabrique automatiquement deux : `nom_2d` et `nom_3d`. Les détails sont en annexe XI.5.

V.3.d Le chef d'orchestre : la boucle de compute_temperature

Le calcul d'un chemin est piloté par une **boucle** dans la fonction `compute_temperature`. À chaque tour, elle appelle la fonction correspondant au mode de transfert en cours (rangée dans `T->func`). Elle s'arrête quand la température est connue (`T->done` vrai). Les fonctions appelées sont présentées dans le tableau 4.

Fonction	Rôle
`XD(conductive_path)`	Fait avancer le chemin par conduction dans un solide
`XD(convective_path)`	Fait avancer le chemin par convection dans un fluide
`XD(radiative_path)`	Fait avancer le chemin par rayonnement (non étudiée ici)
`XD(boundary_path)`	Décide quoi faire quand le chemin arrive sur une interface entre deux milieux

Tableau 4 : Les fonctions appelées par la boucle principale

À une interface solide-fluide, `XD(boundary_path)` appelle `XD(solid_fluid_boundary_picard1_path)`, qui tire au hasard le mode suivant : conduction, convection ou rayonnement. Le chemin poursuit alors dans la fonction choisie, puis revient dans la boucle. La figure 4 montre ce cycle.

V.4 Hack de Stardis : la conduction dans un fluide

Résultat : les chemins thermiques de Stardis peuvent désormais se propager par conduction dans un fluide, lorsque l'utilisateur le demande. Il s'agit d'un prototype destiné aux tests (on parle de « hack » : une modification rapide, non officielle, pour vérifier une idée).

V.4.a Comment indiquer à Stardis qu'un fluide conduit la chaleur ?

Il fallait un moyen de dire, dans le fichier de modèle, « dans ce fluide, fais de la conduction avec telle conductivité λ ». Plutôt que de modifier le format des fichiers d'entrée, nous avons choisi de **cacher cette information dans les décimales du coefficient de convection** h de la paroi qui entoure le fluide (tableau 5).

Décimales de h	**Signification**
3e, 4e et 5e = `101`	Code secret : « conduction dans ce fluide »
6^{e}	Signe de l'exposant : 0 = multiplier, 1 = diviser
7^{e}	Puissance de 10 à appliquer
8e à 11^{e}	Chiffres de λ (lus comme un nombre à 3 décimales)

Tableau 5 : Codage de la conductivité dans le coefficient h

Exemple : avec $h = 10{,}00101122500$, le code `101` est présent, la mantisse vaut 2,500, et on la divise par 10^2 : $\lambda = 0{,}025$ W·m^{-1}·K^{-1} (ordre de grandeur de la conductivité de l'air). Le coefficient h ne change que dans ses dernières décimales, donc l'astuce ne perturbe pas les autres calculs.

V.4.b Vue d'ensemble

La figure 4 montre la boucle de calcul après modification. Les cases en couleur sont les fonctions que j'ai ajoutées ; le reste existait déjà. À chaque passage sur une paroi, Stardis tire un mode de transfert. Si c'est la convection et que h contient le code 101, le chemin est décalé dans le fluide puis part en conduction ; sinon tout se passe comme avant. La boucle recommence tant que la température n'est pas connue.

V.4.c Étape A : détecter le code dans la fonction de convection

Pour lire h, il faut connaître un point de la paroi du fluide. Or la fonction `XD(convective_path)` tire justement un point au hasard sur la paroi (par un lancer de rayon dans une direction aléatoire, résultat rangé dans `rwalk->XD(hit)`). J'ai réutilisé ce premier tirage, puis lu h avec

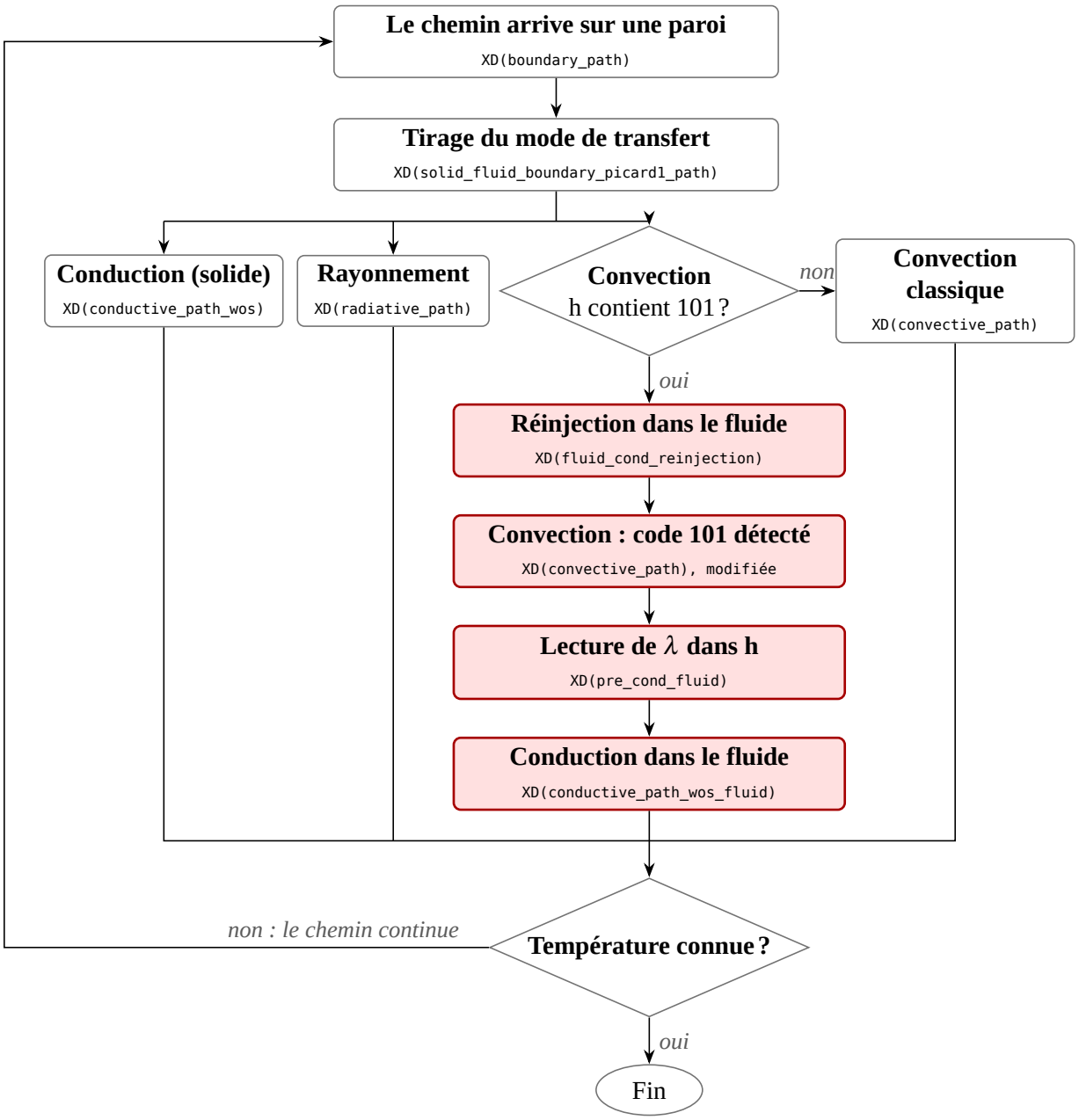


Figure 4 : Boucle des appels de fonctions orchestrant le chemin thermique ; en rouge, les fonctions ajoutées pendant mon stage

les fonctions existantes :

- `scene_get_interface(...)` retrouve la paroi à partir du triangle touché (dans Stardis, toutes les parois sont découpées en triangles, appelés « primitives ») ;
- `XD(setup_interface_fragment)(...)` prépare un « fragment », c'est-à-dire les informations sur ce point de paroi ;
- `interface_get_convection_coef(...)` renvoie *h*.

Une subtilité : ce tirage déplace le chemin, et modifie donc la fiche `rwalk`. Or la conduction doit partir de là où le chemin était *avant* ce tirage. J'ai donc fait une copie de sauvegarde au tout début de la fonction (`struct rwalk rwalk_init = *rwalk;`), lancé la conduction depuis cette copie, puis recopié le résultat dans `rwalk` (figure 5).

```
/* le code 101 est-il présent dans h ? */
if (((int)(hc * 100000)) % 1000 == 101) {
  /* lit λ dans h et lance la conduction depuis la copie */
  XD(pre_cond_fluid)(scn, ctx, &rwalk_init, rng, T, &hc);
  /* reprend l'état final du chemin */
  *rwalk = rwalk_init;
  break;
}
```

Figure 5 : Appel dans XD(convective_path) à la fonction qui lit λ et lance la conduction, puis remplacement de rwalk

La fonction `XD(pre_cond_fluid)` (code complet en annexe XI.6) décode λ à partir de *h*, puis appelle la fonction de conduction dans le fluide.

V.4.d Étape B : une fonction de conduction adaptée aux fluides

J'ai créé `XD(conductive_path_wos_fluid)` en copiant `XD(conductive_path_wos)`, la fonction de conduction des solides (*wos* signifie *walk on spheres*, la méthode de marche aléatoire utilisée). Trois changements ont suffi :

- vérifier que le chemin est bien dans un **fluide** et non plus dans un solide : `ASSERT(sdis_medium_get_type(mdm) == SDIS_FLUID);;`
- remplacer la lecture des propriétés du solide par une nouvelle fonction, `fluid_cond_get_properties`, qui remplit la même fiche de propriétés avec celles du fluide (masse volumique, capacité thermique, température) et avec le λ décodé ;
- fixer le paramètre δ (delta), la taille caractéristique des sauts de la marche aléatoire. Il vaut en général environ 1/20 de la taille du milieu. Mes tests portant sur un cube de 1 m de côté, je l'ai fixé à 0,05 directement dans le code : **il faudra le modifier pour un**

autre milieu.

V.4.e Étape C : la réinjection, la partie la plus longue

À ce stade, je pensais avoir fini, mais le programme plantait.

Le problème. Quand un chemin arrive sur une paroi et part en conduction, il ne peut pas démarrer *exactement* sur la paroi : les erreurs d'arrondi des calculs le feraient sortir du milieu. Stardis le décale donc d'une petite distance vers l'intérieur : c'est la **réinjection**. Elle était prévue avant la conduction dans un solide, mais pas avant la convection. Comme ma conduction dans le fluide est appelée depuis la convection, le chemin n'était jamais décalé.

La solution. Dans `XD(solid_fluid_boundary_picard1_path)`, lorsque le tirage choisit la convection, je teste h. S'il contient le code `101`, je décale le chemin d'une distance δ, perpendiculairement à la paroi, vers le fluide. Il fallait pour cela trois informations :

1. **la distance** : $\delta = 0{,}05$;
2. **la position du chemin** : dans `rwalk` ;
3. **la direction perpendiculaire à la paroi** : la plus difficile à trouver. Elle est en fait déjà rangée dans le fragment `frag`, sous forme d'un vecteur normal (perpendiculaire) orienté vers la face avant de la paroi (`SDIS_FRONT`). Si le fluide est du côté avant, on garde ce vecteur ; sinon, on prend son opposé.

J'ai rangé ces informations dans les structures déjà utilisées pour la réinjection dans les solides (`solid_reinjection_args` et `reinjection_step`). La réinjection dans un solide est plus complexe (décalage à 45° et gestion des coins) ; je n'utilise que les trois champs utiles, mais j'ai gardé les autres pour qu'un futur développeur puisse étendre le code à des géométries plus complexes. La nouvelle fonction `XD(fluid_cond_reinjection)` appelle simplement la fonction existante `XD(move_pos)`, qui ajoute à la position le vecteur direction multiplié par la distance. Le code complet est en annexe XI.6.

Une difficulté du langage C. Dans `solid_reinjection_args`, le champ `reinjection` est déclaré `const` : on ne peut pas modifier ce qu'il désigne. La solution a été de créer une nouvelle structure `reinjection_step`, de la remplir, puis de faire pointer le champ vers elle. On ne modifie pas l'objet désigné, on change d'objet.

V.4.f Le dernier bug : un « point de contact » oublié

Même avec la réinjection, des erreurs apparaissaient dans `XD(check_enclosure_consistency)`, la fonction qui vérifie que le chemin se trouve bien dans le milieu qu'il croit occuper.

- **Symptôme :** le chemin se retrouvait hors de son enceinte, ce qui est impossible physiquement.

- **Indice :** en visualisant les chemins dans Paraview, j'ai vu qu'au moment de l'erreur, le chemin n'était pas sur une paroi alors que la fonction d'interface était appelée.
- **Cause :** la fiche `rwalk` garde en mémoire le dernier point de paroi touché (`hit`). Pendant une conduction, quand le chemin touche une paroi, ce point est mis à jour. Mais il n'était pas effacé quand le chemin repassait en convection. À la conduction suivante, la fonction croyait toucher une paroi dès le départ et appelait la fonction d'interface au mauvais endroit.
- **Correction :** une seule ligne, après la réinjection : `rwalk->XD(hit) = SXD_HIT_NULL;` (« aucun point de paroi touché »).

V.4.g Bilan de l'étape 1 et limites

Avec ces modifications, l'étape 1 est terminée et fonctionne : les chemins thermiques peuvent passer en conduction dans un fluide. Les limites connues du prototype sont :

- δ est écrit en dur (0,05), adapté seulement à un cube de 1 m ;
- la réinjection ne gère que des géométries simples (pas de coins ni de parois très proches) ;
- l'information est cachée dans les décimales de h, ce qui est pratique pour tester mais devra être remplacé par une vraie option dans une version officielle.

L'étape 2, le mode de transfert « en humidité », a été commencée pendant la dernière semaine du stage et pourra être poursuivie après.

V.5 Conseils pour les futurs arrivants sur Stardis

Cette partie répond au second objectif du stage : aider les prochaines personnes qui modifieront Stardis à éviter les difficultés que j'ai rencontrées (tableau 6).

Difficulté rencontrée	Conseil
J’ai modifié du code avant de comprendre l’architecture générale ; je n’avais pas vu que les fonctions de chemin sont appelées dans une boucle	Commencer par repérer la boucle de `compute_temperature` et les fonctions `XD(...)_path` avant de toucher au code
J’ai passé du temps sur des parties du code peu utiles, et découvert tard des parties essentielles	Demander dès le début aux développeurs quelles fonctions sont concernées par la modification
Erreurs de compilation incomprises	Apprendre d’abord comment un projet C est découpé en fichiers `.c` et `.h` et comment ils sont reliés (annexe XI.4)
Notation des pointeurs (`*`, `&`, `->`) déroutante	S’entraîner sur de petits exemples avant d’attaquer Stardis
Fonctions `XD(...)` introuvables sous ce nom	Savoir que la macro fabrique `nom_2d` et `nom_3d` (annexe XI.5)
Champ `const` impossible à modifier	Créer une nouvelle structure et faire pointer le champ vers elle (partie V.4)
Bug sans message clair (le point `hit` non effacé)	Visualiser les chemins dans Paraview pour voir où le chemin se trouve au moment de l’erreur
Rester bloqué seul pendant des heures	Poser la question tôt : l’équipe est très disponible
Consignes des TP parfois fausses ou floues	Utiliser la version corrigée du TP 1, et signaler les erreurs rencontrées

Tableau 6 : Difficultés rencontrées et conseils pour les suivants

VI Enjeux sociétaux et environnementaux

Simuler précisément la chaleur dans des systèmes complexes aide à répondre à plusieurs défis actuels. Les méthodes de Monte Carlo y ont un atout particulier : elles traitent des problèmes à plusieurs échelles et à plusieurs phénomènes couplés, là où les méthodes classiques peinent (tableau 7).

Enjeu	Apport de la simulation thermique
Efficacité énergétique	Repérer les pertes de chaleur et optimiser bâtiments, procédés industriels, électronique et transports
Climat	Moins d'énergie consommée pour chauffer et refroidir, donc moins d'émissions de gaz à effet de serre
Villes et réchauffement	Étudier les îlots de chaleur urbains, où interviennent bâtiments, matériaux, air et soleil à la fois
Transition énergétique	Améliorer la gestion thermique des batteries, du stockage d'énergie et des panneaux solaires, donc leur rendement et leur durée de vie
Confort et coût de la vie	Concevoir des logements plus confortables et moins chers à chauffer
Sécurité	Anticiper les surchauffes en aéronautique, spatial, nucléaire et électronique de puissance
Coûts de développement	Remplacer une partie des prototypes physiques par des simulations, et économiser des matériaux

Tableau 7 : Enjeux de la simulation thermique

À mon échelle, élargir les capacités de Stardis permet de modéliser de nouvelles situations, comme la neige et la vapeur d'eau, jusqu'ici inaccessibles.

VII Analyse réflexive

VII.1 Au cours de mon stage

Sur le plan technique. C'était mon premier projet informatique de grande taille, avec un code réparti sur des dizaines de fichiers, ce qui m'a d'abord déstabilisé. J'ai commencé à modifier le code sans avoir une vue d'ensemble suffisante. Par exemple, je n'avais pas compris tout de suite que les fonctions de chemin étaient appelées dans une boucle. J'ai aussi étudié des parties du code peu utiles et découvert tard des parties essentielles. J'en retiens qu'il faut **comprendre l'architecture d'un programme avant de le modifier** : on gagne ensuite beaucoup de temps.

Sur le plan humain. Au GREPHE, les portes sont ouvertes et chacun prend le temps de répondre. Dès le premier jour, Richard m'a dit de venir l'interrompre dès que j'avais besoin d'aide. J'ai découvert que, dans un laboratoire, les échanges informels comptent beaucoup, et qu'il vaut mieux demander l'avis d'une personne expérimentée que rester bloqué seul pendant des heures. Plusieurs de mes problèmes techniques ont été résolus grâce aux conseils de l'équipe : un projet complexe repose autant sur la collaboration que sur les compétences individuelles.

VII.2 Au sein de mon projet professionnel

J'hésitais entre la physique et l'informatique : j'aime l'informatique depuis la spécialité NSI au lycée, et la physique depuis mes études scientifiques. Ce stage m'a aidé à trancher :

- mes principales difficultés venaient de lacunes en informatique, pas en physique ;
- j'ai pris beaucoup de plaisir à travailler sur un code aussi vaste que Stardis ;
- Christophe et Vincent, formés en informatique, développent un logiciel fondé sur une physique complexe : l'informatique ouvre la porte à de nombreux domaines, dont la physique.

Je souhaite donc m'orienter vers le **parcours SN (Sciences du Numérique) de l'ENSEEIHT**, pour appliquer l'informatique à des problèmes scientifiques.

VIII Conclusion

En cinq semaines, j'ai d'abord acquis les bases qui me manquaient : la physique des méthodes de Monte Carlo, la programmation en C et l'utilisation de Linux. Je les ai ensuite appliquées pour atteindre l'objectif principal du stage : **permettre aux chemins thermiques de Stardis de se propager par conduction dans un fluide.** Cette modification fonctionne et servira à tester un modèle de neige et de vapeur d'eau. Pendant la semaine restante, je commence l'étape 2, le mode « en humidité », qui pourra être poursuivie après mon départ.

Cette première expérience dans une équipe de recherche m'a permis de développer :

- ma capacité à m'intégrer dans un environnement de travail réel ;
- ma capacité à synthétiser une information nouvelle et à identifier rapidement le besoin ;
- mes compétences techniques en informatique et en physique ;
- mes capacités de communication, de rédaction et de restitution.

Elle m'a convaincu que j'aime autant l'informatique que la physique, et oriente mon projet vers des études d'informatique appliquée à des domaines variés.

IX Remerciements

Merci à **Stéphane Blanco** de m'avoir accueilli au GREPHE et d'avoir répondu à mes questions tout au long du stage. Merci à **Richard Fournier** de m'avoir suivi et d'avoir pris tout le temps nécessaire pour me former aux méthodes de Monte Carlo. Merci à **Mégane Bati** pour son aide sur les TP Stardis. Merci à **Christophe Coustet** pour les nombreuses notions de programmation transmises, notamment en C. Merci à **Vincent Forest** pour les concepts du

langage C et ses réponses à mes questions sur Stardis.

X Références Bibliographiques

- Laboratoire LAPLACE, présentation générale : `www.laplace.univ-tlse.fr/presentation-generale/` (consulté le 29/05/2026)
- Université Toulouse III, Laboratoire plasma et conversion d'énergie : `www.univ-tlse3.fr/structures-de-recherche/laboratoire-plasma-et-conversion-d-energie` (consulté le 29/05/2026)
- GREPHE, Groupe de Recherche Énergétique, Plasma et Hors Équilibre : `www.laplace.univ-tlse.fr/grephe/` (consulté le 29/05/2026)
- EDStar, démonstrateur et travaux pratiques : `www2.edstar.cnrs.fr` (consulté le 29/05/2026)

XI Annexes

XI.1 Liste des tâches effectuées

- relecture et correction des travaux pratiques Stardis (TP 1) ;
- modification de Stardis pour la conduction dans un fluide (étape 1 terminée) ;
- début de l'étape 2, le mode « en humidité » ;
- participation aux réunions de réflexion sur ma modification, et aux répétitions d'oral d'autres stagiaires.

XI.2 Glossaire et nomenclature

Terme	**Définition simple**
Chemin thermique	Parcours aléatoire qui part du point où l'on cherche la température et remonte le temps jusqu'à une température connue
Monte Carlo	Méthode qui estime une moyenne en répétant une expérience aléatoire un grand nombre de fois
Conduction	Transfert de chaleur à travers la matière, sans mouvement de celle-ci
Convection	Transfert de chaleur par un fluide en mouvement
Rayonnement	Transfert de chaleur par la lumière (visible ou infrarouge)
Interface	Frontière entre deux milieux, par exemple entre un solide et un fluide
Primitive	Petit triangle ; toutes les parois de Stardis sont découpées en triangles
Condition initiale	Température connue au début de la simulation
Réinjection	Petit décalage du chemin vers l'intérieur d'un milieu, pour qu'il ne démarre pas exactement sur la paroi
Hack	Modification rapide et non officielle d'un logiciel, pour tester une idée
Structure (C)	Ensemble de variables regroupées sous un même nom, comme une fiche
Pointeur (C)	Variable qui contient l'adresse en mémoire d'une autre donnée
Macro (C)	Règle de remplacement de texte appliquée avant la compilation

Symbole	**Grandeur**	**Unité**
θ	Température (θ_F : fluide, θ_S : solide)	K
h ou h_c	Coefficient d'échange par convection	$W \cdot m^{-2} \cdot K^{-1}$
λ	Conductivité thermique	$W \cdot m^{-1} \cdot K^{-1}$
ρ	Masse volumique	$kg \cdot m^{-3}$
c	Capacité thermique massique	$J \cdot kg^{-1} \cdot K^{-1}$
V	Volume de fluide	m^3
ν, $\hat{\nu}$	Taux d'échange fluide-paroi, et taux majorant utilisé pour la méthode de rejet	s^{-1}
δ	Taille caractéristique des sauts en conduction, et distance de réinjection	m
$\vec{x}$, t	Position, temps	m, s

XI.3 Fiche de synthèse

prepa t² — Prépa Toulouse Transitions

Méthode de Monte Carlo pour l'étude des transferts thermiques



Laboratoire Laplace - Bâtiment 3R1, 325 Rue Sébastienne Guyot, Université Paul Sabatier - 31400 Toulouse

NOM Prénom	Parcours thématique
Cabanillas David	Physique

Laplace

OBJECTIFS DU STAGE

- S'approprier toute la physique derrière la Méthode de Monte-Carlo
- « Hacker » Stardis, le logiciel de simulation statistique de modèles physiques couplées, c'est à dire modifier le code source de manière non conventionnelle pour pouvoir constater le résultat d'une modification, pour déterminer si une modification officielle peut être pertinente.

MISSIONS EFFECTUÉES ET RÉSULTATS

- Relecture des Travaux Pratiques sur Stardis et modification des erreurs et des éléments peu clairs, afin qu'ils puissent être utilisés lors de la prochaine présentation, mais aussi pour que de futurs élèves ou novices dans le domaine puissent les réaliser.
- « Hack » de Stardis permettant aux chemins thermiques de passer en conduction dans un fluide si on le décide. Je n'ai pas encore réussi à aboutir.
- Participation à des réunions de réflexion sur la réalisation de mon hack. Participation à des présentations d'autres stagiaires réalisant leur oral de stage, en donnant mon avis sur ce qu'ils devraient changer.

COMPÉTENCES DEVELOPPÉES

- Maîtrise des commandes linux pour naviguer dans une distribution avec peu d'interface graphique comme edix
- Acquisition de compétences en programmation C à travers la modification et la compréhension d'un code source complexe. Maîtrise de l'organisation de projets comportant de multiples fichiers .c et .h, utilisation des pointeurs et gestion des valeurs NULL, ainsi que compréhension du fonctionnement global d'un logiciel existant.
- Gestion de projets git
- Compréhension du lien mathématique entre la thermodynamique traditionnelle et la thermodynamique probabiliste vu sous forme de chemins thermiques par la méthode de Monte-Carlo.

ENJEUX AUTOUR DES TES

Le logiciel sur lequel j'ai travaillé permet de modéliser avec précision les transferts de chaleur dans des environnements complexes grâce à des méthodes probabilistes de Monte-Carlo. Ces simulations sont notamment utiles pour l'étude de problématiques liées à l'efficacité énergétique des bâtiments, à l'optimisation des systèmes thermiques ou encore à la compréhension des îlots de chaleur urbains. En contribuant à l'amélioration du code source du logiciel, j'ai participé au développement d'outils permettant une meilleure connaissance et une meilleure maîtrise des phénomènes thermiques, enjeux importants dans le contexte du changement climatique et de la réduction des consommations énergétiques.

XI.4 Notions du langage C

Pointeurs. Un pointeur contient une adresse en mémoire. Il ne faut pas confondre l'adresse et ce qui s'y trouve :

```
void f(struct rwalk* rwalk);  /* f attend l'ADRESSE d'une structure rwalk */
f(&ma_marche);                /* & donne l'adresse de ma_marche */
/* dans f : rwalk est l'adresse, *rwalk est la structure elle-même,
   rwalk->vtx est le champ vtx de la structure pointée */
```

Fichiers .c et .h. Les fonctions sont *définies* (écrites en entier) dans les fichiers `.c` et *déclarées* (seulement leur nom et leurs paramètres) dans les fichiers `.h`. Chaque `.c` est compilé séparément ; pour utiliser une fonction d'un autre fichier, on inclut le `.h` qui la déclare. À l'édition de liens, les morceaux compilés sont assemblés. Exemple de déclaration (figure 6) :

```
extern LOCAL_SYM res_T
conductive_path_wos_3d
  (struct sdis_scene* scn,
   struct rwalk_context* ctx,
   struct rwalk* rwalk,
   struct ssp_rng* rng,
   struct temperature* T);
```

Figure 6 : Déclaration de la fonction conductive_path_wos_3d

static et INLINE. `static` limite une fonction au fichier où elle est écrite (évite les conflits de noms). `INLINE` autorise le compilateur à recopier le code de la fonction à l'endroit de l'appel, pour aller plus vite.

Macros. `#define PI 3.141592` remplace partout `PI` par `3.141592` avant la compilation.

XI.5 Les macros dans Stardis

La macro `XD` transforme une fonction écrite une fois en deux fonctions, 2D et 3D. Par exemple, `XD(convective_path)` produit `convective_path_2d` et `convective_path_3d`, qu'il faut déclarer dans un `.h` (figure 7).

```
extern LOCAL_SYM res_T
convective_path_2d
  (struct sdis_scene* scn, struct rwalk_context* ctx,
   struct rwalk* rwalk, struct ssp_rng* rng, struct temperature* temperature)
   ;

extern LOCAL_SYM res_T
convective_path_3d
  (struct sdis_scene* scn, struct rwalk_context* ctx,
   struct rwalk* rwalk, struct ssp_rng* rng, struct temperature* temperature)
   ;
```

Figure 7 : Déclarations des fonctions convective_path_2d et convective_path_3d dans un fichier .h

Ce sont les appels à d'autres fonctions `XD(...)` à l'intérieur qui rendent les deux versions différentes, par exemple dans `convective_path` (figure 8).

```
enc = scene_get_enclosure(scn, rwalk->enc_id);
if((res = XD(check_enclosure)(scn, rwalk, enc)) != RES_OK) goto error;
if((res = scene_get_enclosure_medium(scn, enc, &mdm)) != RES_OK) goto error;
ASSERT(sdis_medium_get_type(mdm) == SDIS_FLUID);
res = XD(handle_known_fluid_temperature)(ctx, rwalk, mdm, T);
```

Figure 8 : Bout de la fonction convective_path

D'autres macros fabriquent des fonctions en série. Celle de la figure 9 crée une fonction qui renvoie une propriété d'un milieu (par exemple `fluid_get_temperature`).

```
#define DEFINE_MDM_GET_PROP_FUNC(Mdm, Prop)                                  \
  static INLINE double                                                       \
  Mdm##_get_##Prop                                                           \
    (const struct sdis_medium* mdm, const struct sdis_rwalk_vertex* vtx) \
  {                                                                          \
    ASSERT(mdm && mdm->type == MDM_TYPE(Mdm));                               \
    return mdm->shader.Mdm.Prop(vtx, mdm->data);                             \
  }
```

Figure 9 : Définition de la macro DEFINE_MDM_GET_PROP_FUNC

XI.6 Le code ajouté

```
static res_T XD(pre_cond_fluid)(
  struct sdis_scene* scn, struct rwalk_context* ctx, struct rwalk* rwalk,
  struct ssp_rng* rng, struct temperature* T, double *hc)
{
  double lambda, puissance, facteur = 1.0;
  int i;
  res_T res = RES_OK;

  /* 8e à 11e décimales : mantisse de lambda */
  lambda = (((int)(*hc * 100000000000.0)) % 10000) / 1000.0;
  /* 7e décimale : puissance de 10 */
  puissance = ((int)(*hc * 10000000.0)) % 10;
  for (i = 0; i < (int)puissance; i++) facteur *= 10.0;

  /* 6e décimale : 0 = multiplier, 1 = diviser */
  if ((((int)(*hc * 1000000.0)) % 10) == 0) {
    lambda = lambda * facteur;
    res = XD(conductive_path_wos_fluid)(scn, ctx, rwalk, rng, T, &lambda);
    if(res != RES_OK) goto error;
```

```
  } else if ((((int)(*hc * 1000000.0)) % 10) == 1) {
    lambda = lambda / facteur;
    res = XD(conductive_path_wos_fluid)(scn, ctx, rwalk, rng, T, &lambda);
    if(res != RES_OK) goto error;
  } else {
    printf("pas de chance\n");
  }
exit:
  return res;
error:
  goto exit;
}
```

Figure 10 : Fonction XD(pre_cond_fluid), qui décode λ dans h puis lance la conduction (les printf de débogage ont été retirés)

```
static INLINE res_T
fluid_cond_get_properties
  (const struct sdis_medium* mdm, const struct sdis_rwalk_vertex* vtx,
   struct solid_props* props, double *lambda)
{
  ASSERT(mdm && mdm->type == SDIS_FLUID);
  props->lambda = *lambda;
  props->rho = fluid_get_volumic_mass(mdm, vtx);
  props->cp = fluid_get_calorific_capacity(mdm, vtx);
  props->temperature = fluid_get_temperature(mdm, vtx);
  props->delta = 0.05;   /* valeur en dur : 1/20 d'un cube de 1 m */
  props->power = 1;
  props->t0 = fluid_get_t0(mdm);
  return solid_check_properties(mdm->dev, props, vtx->P, vtx->time);
}
```

Figure 11 : Fonction fluid_cond_get_properties

```
if(r < p_conv) {
  T->func = XD(convective_path);

  if (((int)(h_conv * 100000)) % 1000 == 101) {
    double delta_fluid_cond = 0.05;
    struct solid_reinjection_args fluid_cond_reinject_args =
   SOLID_REINJECTION_ARGS_NULL;
    struct reinjection_step reinject_step_fluid_cond = REINJECTION_STEP_NULL;

    fluid_cond_reinject_args.rwalk_ctx = ctx;
    fluid_cond_reinject_args.rwalk = rwalk;
    fluid_cond_reinject_args.rng = rng;
    fluid_cond_reinject_args.T = T;
    fluid_cond_reinject_args.fp_to_meter = scn->fp_to_meter;
```

```
    /* la normale du fragment pointe vers SDIS_FRONT */
    if (fluid_side == SDIS_FRONT) {
      reinject_step_fluid_cond.direction[0] = (float)frag->Ng[0];
      reinject_step_fluid_cond.direction[1] = (float)frag->Ng[1];
      reinject_step_fluid_cond.direction[2] = (float)frag->Ng[2];
    } else {
      reinject_step_fluid_cond.direction[0] = -(float)frag->Ng[0];
      reinject_step_fluid_cond.direction[1] = -(float)frag->Ng[1];
      reinject_step_fluid_cond.direction[2] = -(float)frag->Ng[2];
    }
    reinject_step_fluid_cond.distance = (float)delta_fluid_cond;
    fluid_cond_reinject_args.reinjection = &reinject_step_fluid_cond;

    res = XD(fluid_cond_reinjection)(scn, enc_ids[fluid_side], &
    fluid_cond_reinject_args);
    if(res != RES_OK) goto error;
  }
  rwalk->enc_id = enc_ids[fluid_side];
  /* correction du bug « point de contact oublié » */
  rwalk->XD(hit) = SXD_HIT_NULL;
  rwalk->hit_side = fluid_side;
  break;
}
```

Figure 12 : Partie de XD(solid_fluid_boundary_picard1_path) qui, si h contient le code 101, définit la direction et la distance de réinjection puis appelle la fonction de décalage

```c
res_T
XD(fluid_cond_reinjection)
  (struct sdis_scene* scn, const unsigned fluid_enc_id,
   struct solid_reinjection_args* args)
{
  res_T res = RES_OK;
  XD(move_pos)(args->rwalk->vtx.P,
               args->reinjection->direction,
               args->reinjection->distance);
exit:
  return res;
error:
  goto exit;
}

/* fonction existante : position += direction × distance */
static FINLINE double*
move_pos_3d(double pos[3], const float dir[3], const float delta)
{
  ASSERT(pos && dir);
  pos[0] += dir[0] * delta;
  pos[1] += dir[1] * delta;
  pos[2] += dir[2] * delta;
  return pos;
}
```

Figure 13 : Fonctions XD(fluid_cond_reinjection) et move_pos_3d